Lesson notes · for you

# Testing like an engineer

Programming · Level 4 · Lesson 4

| Time | Materials |
| --- | --- |
| 20 min | A phone with this lesson downloaded; A notebook for tests and review notes |

## Learning goals

- Write unit tests with assert, including edge cases.
- Find and fix a boundary bug in binary search.
- Work test-first, and review code with others.

## Activity: Break my code

1. Warm-up (3 min): List every edge case you can think of for search(a, x). Compare lists with a partner.
2. Trace (7 min): Trace the buggy search (while lo < hi) by hand on search([5], 5). Where does it go wrong?
3. Test (7 min): Pick a new function, such as median(values). Write four asserts, two of them edge cases, before any code.
4. Review (3 min): Swap with a partner and write one comment on their tests: a question about the code, never about the coder.

**Tracing the boundary bug by hand is the moment it clicks: do it on paper before running anything.**

## Discussion

- Why did 90% of professional binary searches have bugs?
- What does watching a test fail first prove?
- How do you make code review feel safe?

## Check yourself

- Write three edge-case asserts for a function that returns the middle value of a list.

**Try it this week**

Before you write your next function, write three asserts for it, one of them at an edge. Watch them fail, then make them pass.

Worksheet

# Testing like an engineer

Programming · Level 4 · Lesson 4

Name _____ Class _____ Date _____

**1** **Working test-first, you add a test for the next small thing, and it passes before you change any code. What does that tell you?**

- ◯ The test checks nothing new, or checks the wrong thing
- ◯ The function is finished
- ◯ You can go straight to refactoring

**2** **This binary search finds a target in the exact middle of a list, but fails at the edges. Fix it so every edge-case test passes.**

```
def search(a, x):
    lo, hi = 0, len(a) - 1
    while lo < hi:
        mid = (lo + hi) // 2
        if a[mid] == x:
            return mid
        elif a[mid] < x:
            lo = mid + 1
        else:
            hi = mid - 1
    return -1
```

Write your program:

**3** **assert top([-5, -2]) == -2 fails: top returns 0. Which line is wrong?**

Circle the line with the mistake. Write it correctly:

```
1  def top(marks):
2      best = 0
3      for m in marks:
4          if m > best:
5              best = m
6      return best
```

**4** **Put the test-first cycle in order.**

- ☐ Write the simplest code that passes it
- ☐ Write a test for the next small thing, and watch it fail
- ☐ Repeat with the next test
- ☐ Tidy the code while every test still passes

**5** **Which are good edge cases for search(a, x)?**

- ☐ A list with one item
- ☐ x is the first or the last item
- ☐ An empty list
- ☐ x is in the middle of a long list

✎ **Think of a function you wrote that you trusted because it "looked right". Which edge case might break it? Write the assert that would find out.**

Check your answers

# Testing like an engineer

Programming · Level 4 · Lesson 4

**1** **The test checks nothing new, or checks the wrong thing**

A new test should fail first (red): that proves it can catch the missing behaviour.

**2**

```
def search(a, x):
    lo, hi = 0, len(a) - 1
    while lo <= hi:
        mid = (lo + hi) // 2
        if a[mid] == x:
            return mid
        elif a[mid] < x:
            lo = mid + 1
        else:
            hi = mid - 1
    return -1
```

while lo < hi skips the last place left to look, when lo equals hi.

**3** **Line 2: `best = marks[0]`**

Starting best at 0 assumes there is always a mark above 0.

**4** **Top to bottom: 2, 1, 4, 3**

Red, green, refactor, repeat. Seeing the test fail first proves it can catch the problem; tidying only on green keeps the code safe.

**5** **An empty list; A list with one item; x is the first or the last item**

Empty, single, first and last are where the lo and hi boundaries go wrong.